Inteligență Artificială

Bogdan Alexe
bogdan.alexe@fmi.unibuc.ro

Radu Ionescu
raducu.ionescu@gmail.com

Informatică, anul II, 2025-2026

Recapitulare – cursul trecut

1. Algoritmul α-β retezare (α-β prunning)
2. Căutare ExpectiMax
3. Probleme de satisfacere de constrângeri
4. Algoritmi de căutare locală
5. Algoritmi genetici

Cuprinsul cursului de azi

1. Calcul evolutiv
2. Algoritmi genetici

Calcul evolutiv – motivație și idee generală

În multe probleme de optimizare:

- spațiul soluțiilor este foarte mare;
- funcția cost/obiectiv prezintă multe minime/maxime locale;

Avem nevoie de o metode care să exploreze simultan mai multe regiuni ale spațiului soluțiilor.

Ideea centrală a **calculului evolutiv**

- menținem o **populație de soluții candidat**;
- evaluăm fiecare soluție printr-o **funcție de fitness**;
- selectăm preferențial soluțiile cele mai bune;
- generăm soluții noi prin variație (**încrucișare și mutație**);
- repetăm procesul până când este îndeplinită o condiție de oprire (calitatea soluțiilor devine satisfăcătoare, am atins un număr maxim de generații, etc)

Calculul evolutiv echilibrează:

- **explorarea** (cautărea de regiuni noi din spațiul soluțiilor);
- **exploatarea** (rafinarea soluțiile bune).

Acest „duet” permite rezolvarea unor probleme unde metodele deterministe sau greedy pot eșua.

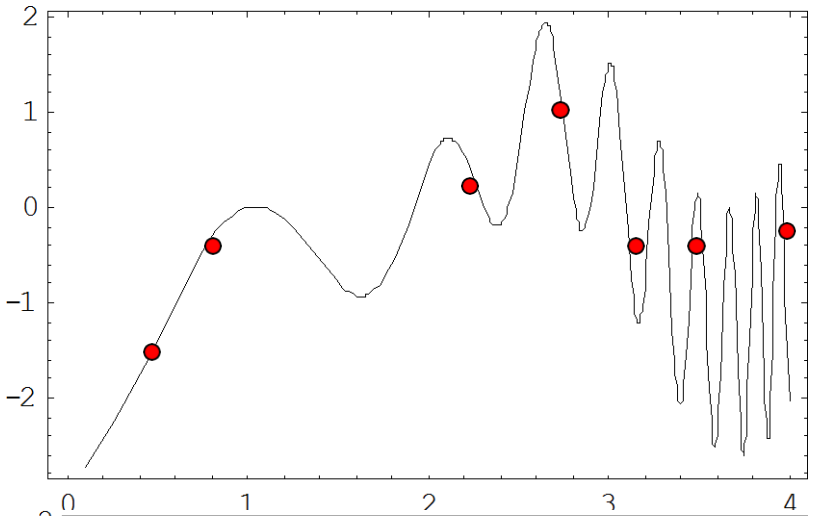
Specificul calculului evolutiv

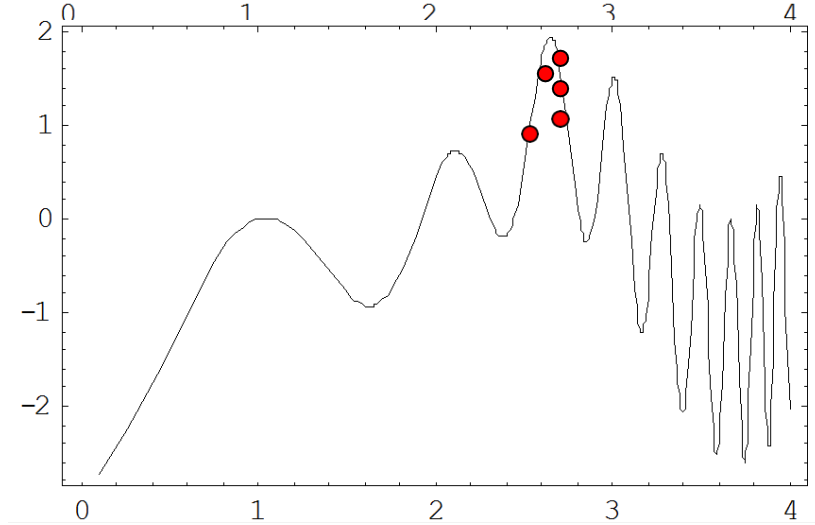
Procesul de căutare se bazează pe două mecanisme principale:

explorare = investigarea diferitelor regiuni din spațiul soluțiilor pentru a descoperi soluții promițătoare

exploatare = rafinarea soluțiilor promițătoare (folosind informațiile acumulate în explorare)

Este necesar un echilibru între explorare și exploatare.





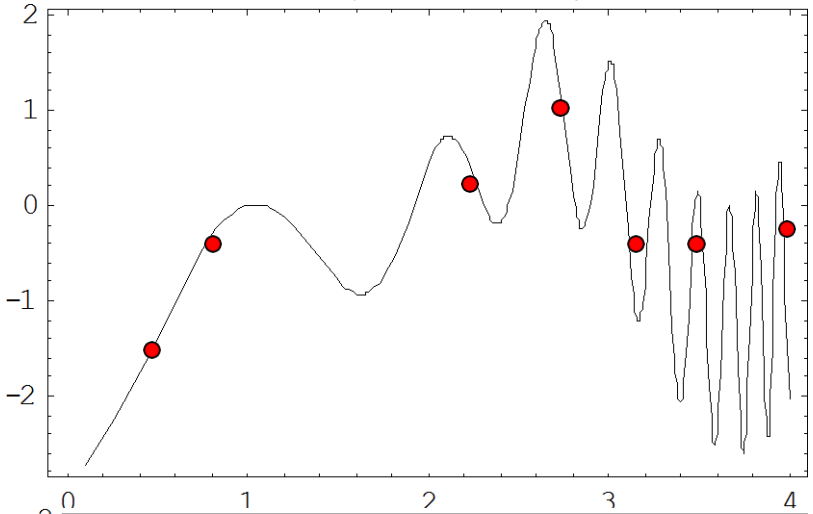
Calculul evolutiv → algoritmi evolutivi

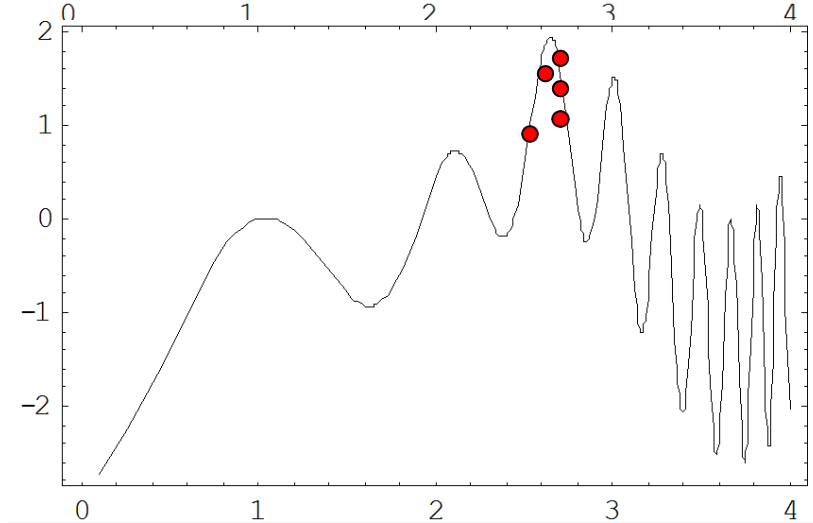
Acest echilibru dintre explorare și exploatare este implementat în practică prin algoritmi evolutivi.

Algoritmul evolutiv este mecanismul concret prin care implementăm explorarea și exploatarea:

- variația (încrucișare + mutație) realizează explorarea
- selecția realizează exploatarea.

încrucișare + mutație





selecția

Structura unui algoritm evolutiv

Inițializare populație (aleatoare)
REPETĂ
    Evaluarea populației (calcul fitness)
    Selecția părinților
    Generarea urmașilor prin încrucișare + mutație
    Evaluarea urmașilor (calcul fitness)
    Selecția supraviețuitorilor (din părinți și/sau urmași)
PÂNĂ CÂND <condiție de oprire>

Tipuri de algoritmi evolutivi

Algoritmii evolutivi diferă în principal prin modul de reprezentare a soluțiilor:

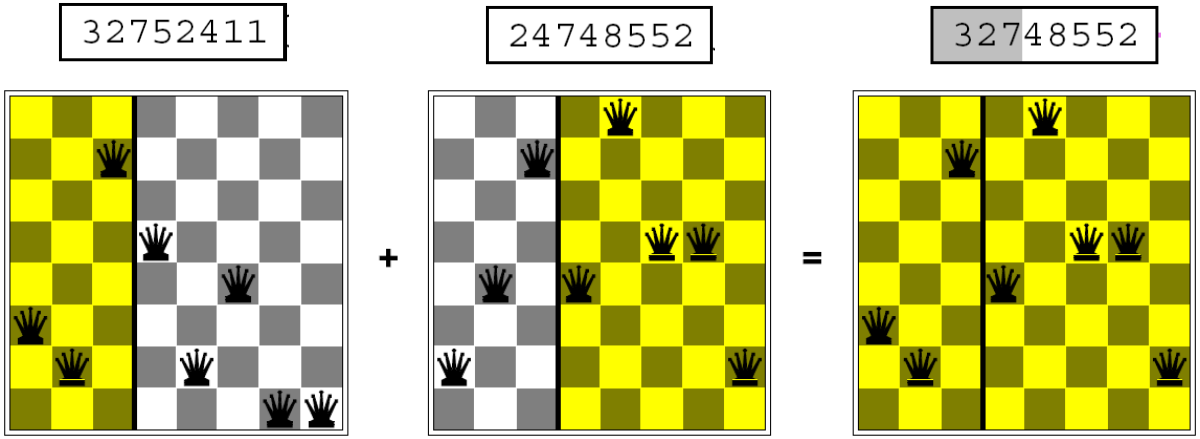
- Șiruri binare (codificare discretă)
  → Algoritmi genetici

- Vectori de valori reale
  → Strategii evolutive

- Structuri de tip mașină de stări / programe
  → Programare evolutivă

- Structuri de tip arbore (expresii, programe)
  → Programare genetică

Algoritmi genetici – exemplu



Populație = mulțime de indivizi (ipoteze, stări, asignări, configurări)

Funcție de evaluare (fitness) = evaluează calitatea unei ipoteze

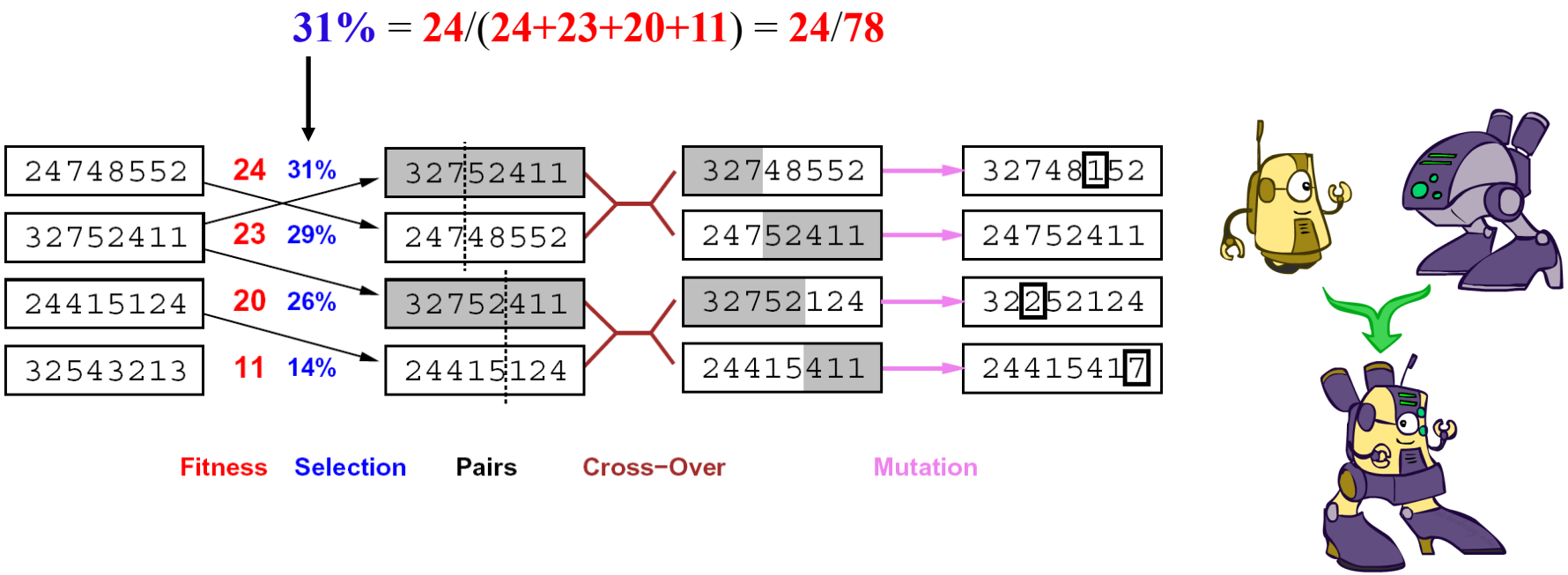
-exemplu în problema de N-regine: numărul de perechi de regine care nu se atacă

f (32752411) = 23 = 28 - 5

f (24748552) = 24 = 28 - 4

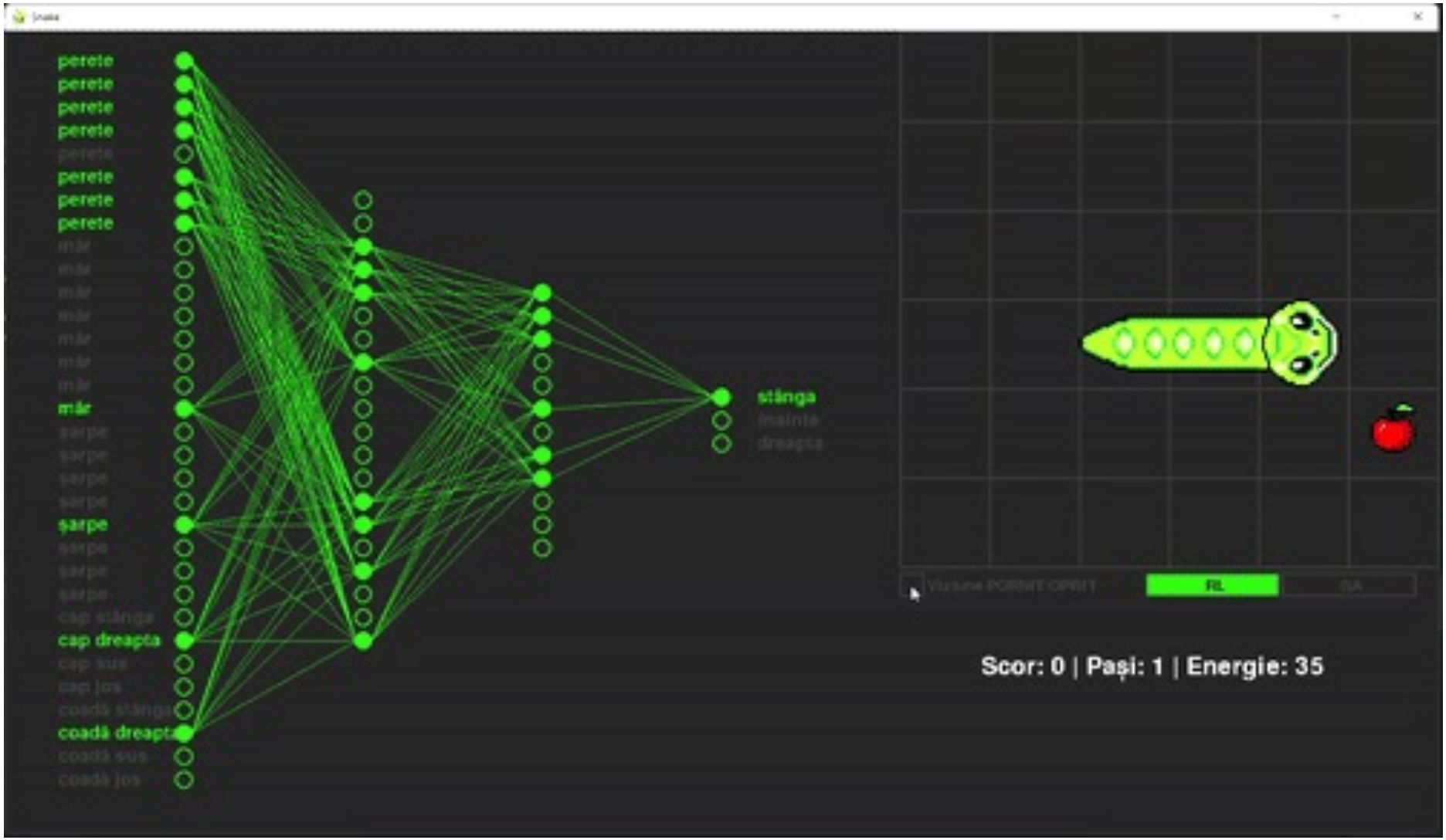
Obține o nouă populație combinând indivizi folosind un proces care se bazează pe o selecție după o anumită distribuție de probabilitate și împerechere bazată pe operațiile de încrucișare (cross-over) și mutație.

Algoritmi genetici – exemplu



- Obține o nouă populație combinând indivizi folosind un proces care se bazează pe o selecție (selection) după o anumită distribuție de probabilitate bazată pe funcția de evaluare (fitness) si împerechere (**pairs**) bazată pe operațiile de **încrucișare (cross-over)** și mutație (mutation)

Algoritmi genetici – exemplu



Algoritmi genetici – exemplu

8	6	9	7	1	3	8	2	5
5	1	2	8	4	6	9	7	3
3	4	7	9	2	5	4	6	1
9	5	4	1	8	2	7	3	6
3	7	1	9	6	8	2	5	4
6	2	8	5	3	4	1	8	9
2	3	6	4	7	1	5	9	8
1	9	3	2	8	5	6	4	7
4	8	5	6	9	7	3	1	2

Algoritmi genetici

Un algoritm genetic este definit de următoarele componente:

- Codificarea soluțiilor (cromozomi)
  – modul în care reprezentăm soluțiile (ex: șiruri binare)

- Funcția de fitness
  – măsoară calitatea unei soluții

- Selecția
  – alege soluțiile mai bune pentru reproducere

- Încrucișarea (crossover)
  – combină informația din doi părinți (explorare)

- Mutația
  – introduce variație aleatorie (explorare)

Codificarea soluțiilor (cromozomilor)

Codificarea reprezintă un element esențial în proiectarea unui algoritm genetic.

• Definește modul în care reprezentăm soluțiile (cromozomi)

• Alegerea codificării depinde de natura problemei

Variante de codificare:

- codificare binară (varianta clasică pentru algoritmi genetici)
- codificare reală (adecvată optimizării în domenii continue)
- codificare specifică (pentru cazul când se caută configurații cu o structură specială – de exemplu, permutări)

Codificare binară

Cromozom = șir de biți

Spațiul de căutare: $\{0,1\}^n$, unde *n* este corelat cu dimensiunea problemei.

Exemple:

1. Problema ONEMAX: determinarea șirului de biți $(x_1 ,\ldots,x_n)$ care maximizează funcția $f(x_1 ,\ldots,x_n) = x_1 + \ldots + x_n$

2. Problema Rucsacului: se consideră un set de obiecte caracterizate de greutățile $(w_1,\ldots,w_n)$ și valorile $(v_1 ,\ldots,v_n)$ și un rucsac de capacitate C; să se determine un subset de obiecte ce pot fi incluse în rucsac astfel încât să nu fie depășită capacitatea acestuia și valoarea totală a obiectelor selectate să fie maximă.

Reprezentarea soluției: $(s_1,\ldots, s_n)$, unde $s_i = 0$ dacă obiectul i nu este selectat și $s_i =1$ dacă obiectul i este selectat

Codificare binară

3. Optimizarea unei funcții definite pe un domeniu continuu.

$$f : [a_1, b_1] \times \ldots \times [a_n, b_n] \to \mathbb{R}$$

$$X = (x_1, \ldots, x_n) \to V = (v_1, \ldots, v_n)$$

$$V = (v_1, \ldots, v_n) \to U = (u_1, \ldots, u_n)$$

$$U = (u_1, \ldots, u_n) \to Y = (y_1, \ldots, y_n, y_{n+1}, \ldots, y_{nr})$$

$$v_i = (x_i - a_i)/(b_i - a_i), v_i \in [0, 1]$$

$u_i = [v_i \cdot (2^r - 1)]$ (u_i este număr natural din $\{0,\ldots 2^r\text{-}1\}$ => poate fi reprezentat în binar pe r poziții)

$(y_{r(i-1)+1}, \ldots, y_{ri})$ = reprezentarea binară a valorii u_i

Codificare binară

Obs. Reprezentarea binară are dezavantajul că valori numerice apropiate au asociate reprezentări binare aflate la distanță Hamming mare (de exemplu $7=(0111)_2$, $8=(1000)_2$)

Soluție: utilizarea codului Gray (valori succesive au asociate reprezentări binare ce diferă într-o singură poziție)

$$(b_1, \ldots, b_r) \rightarrow (g_1, \ldots, g_r)$$

$$g_1 = b_1$$

$$g_i = (b_{i-1} + b_i) \bmod 2$$

Codificare binară

Codificarea Gray:

$$(b_1, \ldots, b_r) \rightarrow (g_1, \ldots, g_r)$$

$$g_1 = b_1$$

$$g_i = (b_{i-1} + b_i) \bmod 2$$

Decodificarea Gray:

$$b_j = (g_1 + \ldots g_j) \bmod 2$$

Nr.	Binar	Gray
0	000	000
1	001	001
2	010	011
3	011	010
4	100	110
5	101	111
6	110	101
7	111	100

Codificare specifică

Codificare adaptată problemei de rezolvat

Exemplu: codificare de tip permutare

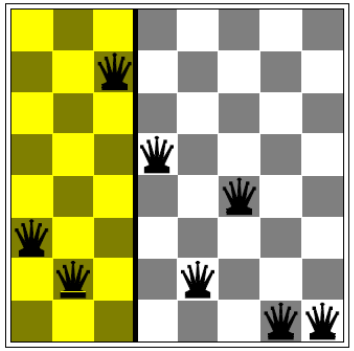
$(s_1,s_2,\ldots,s_n)$, $s_i \in \{1,\ldots,n\}$, $s_i \neq s_j$ pt. orice $i \neq j$

Problema: problema celor N-regine

s_i = linia pe care punem regina de pe coloana i

Observație Prezintă avantajul că asigură implicit satisfacerea restricțiilor

32752411

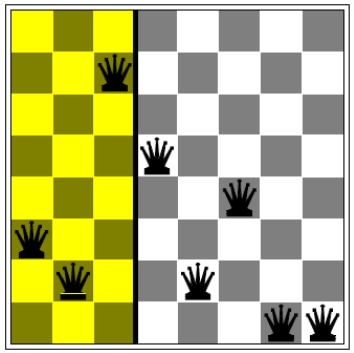


Exemple de funcții de fitness

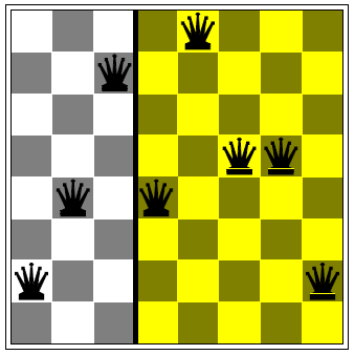
Funcția de fitness (evaluare, scor, adecvare)

- este o măsură a calității unui element al populației
- cu cât valoarea sa este mai mare cu atât va fi mai mare probabilitatea ca elementul respectiv să supraviețuiască (direct sau prin intermediul urmașilor săi)

32752411



24748552



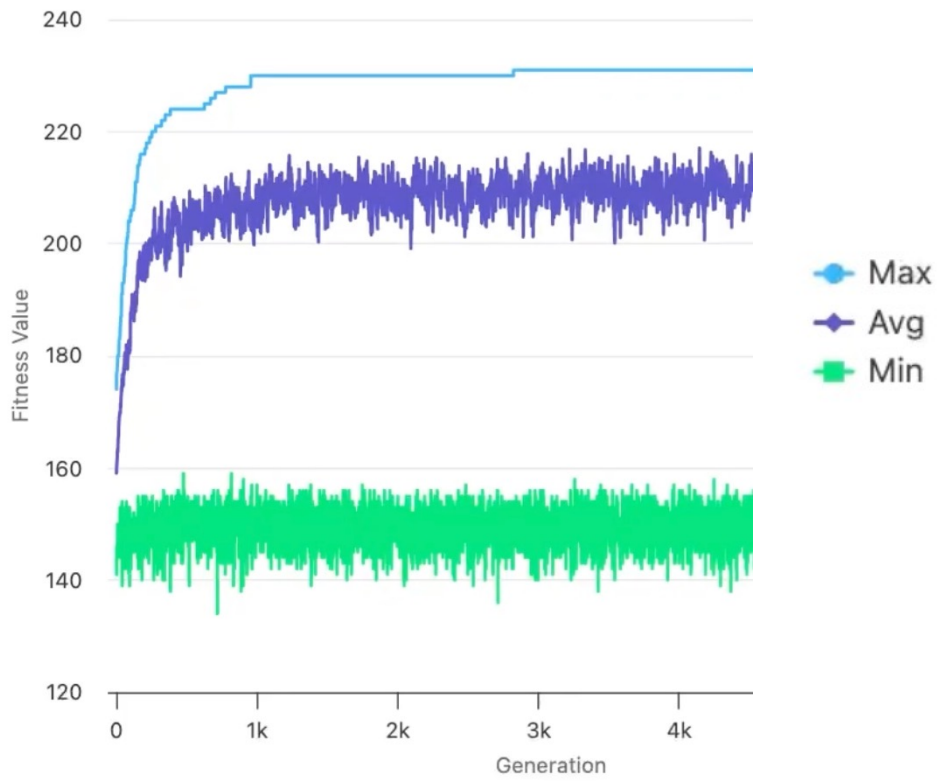
Exemplu în problema de N-regine: numărul de perechi de regine care nu se atacă

f (32752411) = 23 = 28 - 5

f (24748552) = 24 = 28 - 4

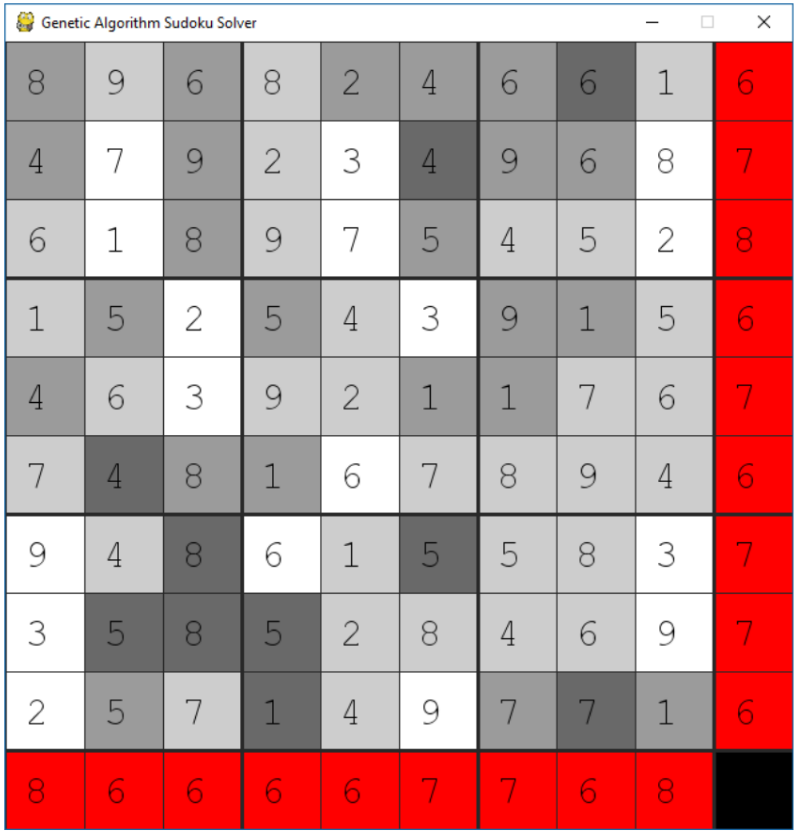
Exemple de funcții de fitness

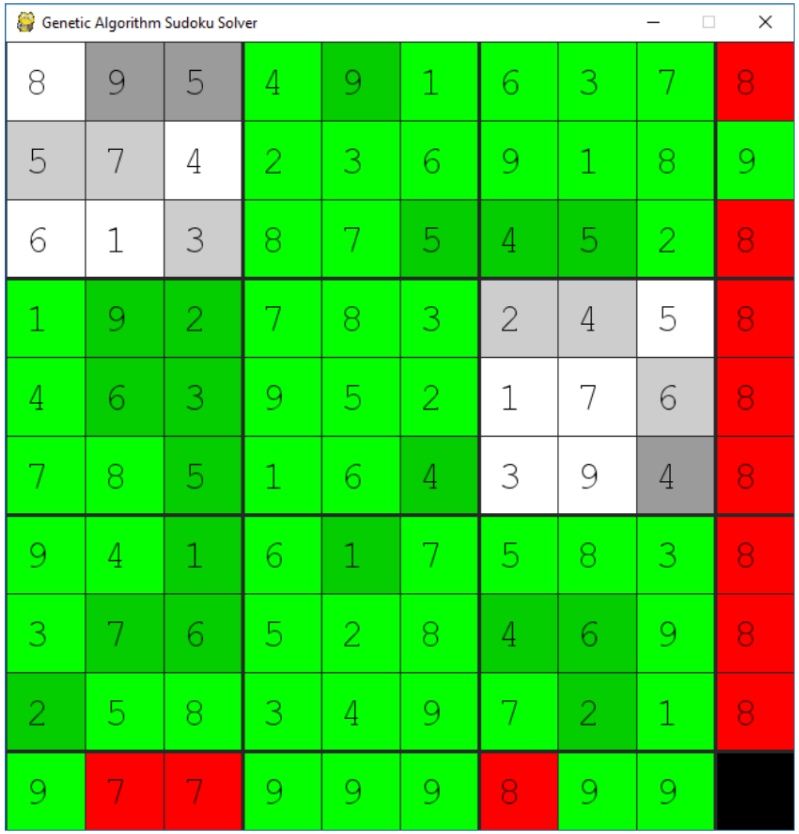
8	6	9	7	1	3	8	2	5
5	1	2	8	4	6	9	7	3
3	4	7	9	2	5	4	6	1
9	5	4	1	8	2	7	3	6
3	7	1	9	6	8	2	5	4
6	2	8	5	3	4	1	8	9
2	3	6	4	7	1	5	9	8
1	9	3	2	8	5	6	4	7
4	8	5	6	9	7	3	1	2



Celulele fără conflicte (alb) primesc scor maxim (3).
Celulele care încalcă constrângeri (nuanțe de roșu) sunt penalizate (scor 2, 1 sau 0, în funcție de numărul conflictelor).
Funcția de fitness este suma scorurilor pe toate cele 81 de celule; valoarea maximă este 243.

Exemple de funcții de fitness





Pentru fiecare linie și coloană se calculează numărul de valori distincte. Scorul total (fitness) este suma acestor valori.

Selecția

Scop:

- stabilirea indivizilor din populația curentă care vor participa la constituirea urmașilor (selecția părinților)
- stabilirea indivizilor din populația de părinți și/sau urmași care vor face parte din generația viitoare (selecția supraviețuitorilor)

Inițializare populație (aleatoare)
REPETĂ
    Evaluarea populației (calcul fitness)
    Selecția părinților
    Generarea urmașilor prin încrucișare + mutație
    Evaluarea urmașilor (calcul fitness)
    Selecția supraviețuitorilor (din părinți și/sau urmași)
PÂNĂ CÂND <condiție de oprire>

Selecția

Scop:

- stabilirea indivizilor din populația curentă care vor participa la constituirea urmașilor (selecția părinților)
- stabilirea indivizilor din populația de părinți și/sau urmași care vor face parte din generația viitoare (selecția supraviețuitorilor)

Principiu:

- elementele care au valoare de fitness mare au mai multe șanse să fie selectate

Mecanisme de selecție:

- selecție proporțională
- selecție pe baza rangurilor
- selecție de tip turneu
- selecție prin trunchiere

Selecția proporțională

Populația curentă: $P=(x^1,\ldots,x^m)$

Valori funcție de adecvare:

$(F_1,\ldots,F_m)$

Probabilități de selecție:

$p_i=F_i/(F_1+\ldots+F_m)$

Obs. Dacă F_i nu sunt strict pozitive se modifică:

$F'_i=F_i-\min(F_i)+eps$

Etape:

a) Calculul probabilităților de selecție

b) Generarea de valori aleatoare în conformitate cu distribuția

1	2	…	m
p_1	p_2	…	p_m

Variante de implementare:

(i) “Roulette Wheel”

(ii) “Stochastic Universal Sampling” (SUS)

Selecția proporțională - RW

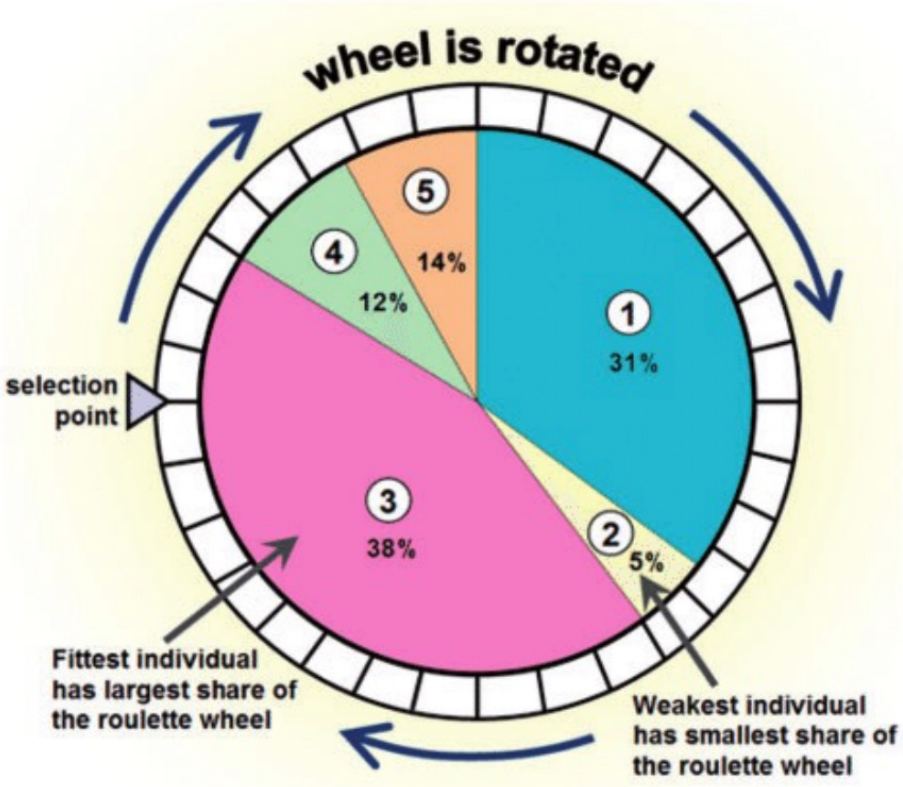
Roulette Wheel (metoda ruletei)

- Se consideră o ruletă divizată în m sectoare de arii proporționale cu probabilitățile de selecție

- Se rotește ruleta și numărul de ordine al sectorului în dreptul căruia se află indicatorul la oprirea ruletei reprezintă elementul din populație care se va selecta

Exemplu:

1	2	3	4	5
0.31	0.05	0.38	0.12	0.14



Selecția proporțională - RW

Implementare:

```
Ruleta (p[1..m])
    i:=1
    s:=p[1]
    u:=random(0,1)
    while s<u do
        i:=i+1
        s:=s+p[i]
    endwhile
    Return i
```

Obs.

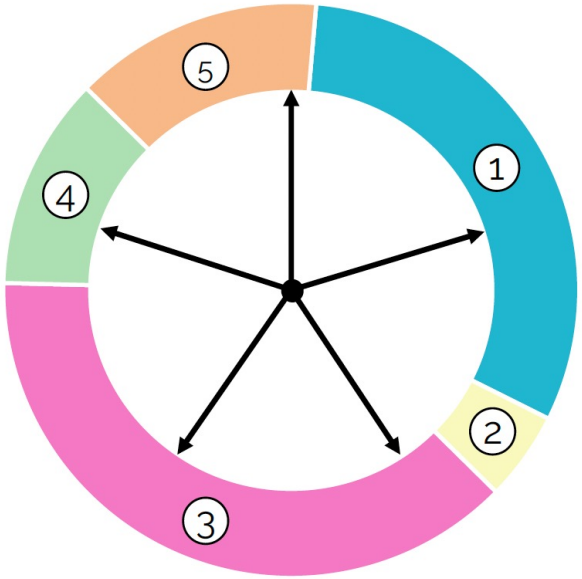
1. Algoritmul alăturat implementează metoda inversării funcției de repartiție
2. La o rulare generează indicele unui element
3. Pentru selectarea mai multor elemente ar trebui aplicat algoritmul de mai multe ori.

   Pentru a eficientiza procesul se poate utiliza varianta SUS (Stochastic Universal Sampling) – permite generarea mai multor valori într-o singură rulare

Selecția proporțională - SUS

Stochastic Universal Sampling

Idee:



Obs: k reprezintă numărul de elemente ce trebuie selectate

c[1..m] va conține nr. de copii ale fiecărui element

Algoritm:

```
SUS(p[1..m],k)
   u:=random(0,1/k)
   s:=0
   for i:=1,m  do
        c[i]:=0
        s:=s+p[i]
        while u<s do
             c[i]:=c[i]+1
             u:=u+1/k
         endwhile
   endfor
   Return c[1..m]
```

Selecția proporțională - SUS

Stochastic Universal Sampling

Idee:

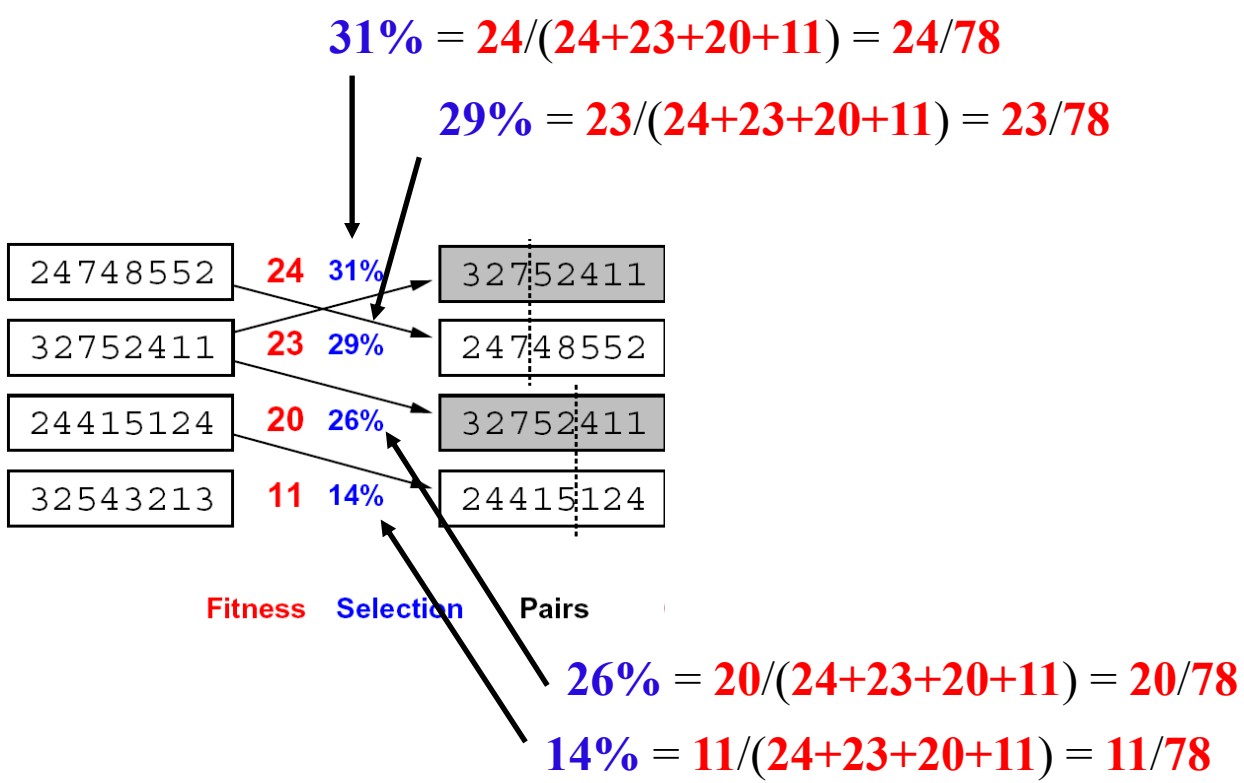
Algoritm:

```
SUS(p[1..m],k)
   u:=random(0,1/k)
   s:=0
   for i:=1,m  do
       c[i]:=0
       s:=s+p[i]
       while u<s do
           c[i]:=c[i]+1
           u:=u+1/k
       endwhile
   endfor
   Return c[1..m]
```

Avantaje:

- distribuție mult mai stabilă și mai puțin aleatorie decât ruleta.
- indivizii cu fitness mare primesc în mod consistent numărul proporțional de copii.
- reduce riscul ca indivizii buni să fie "săriți" din cauza hazardului.

Selecția proporțională

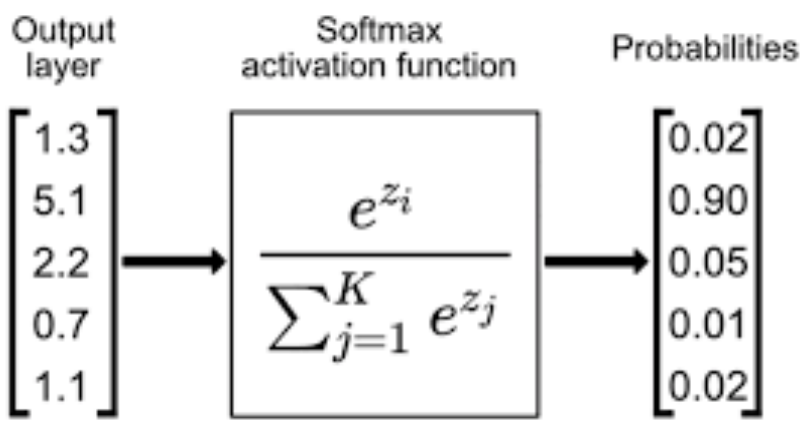


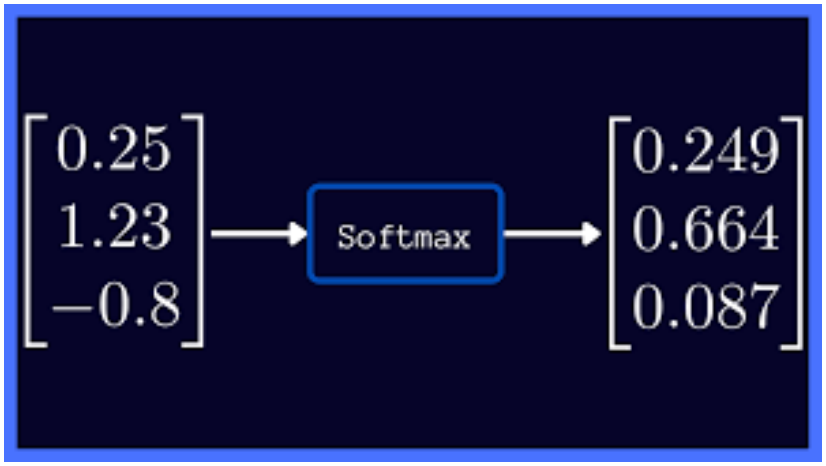
- Obține o nouă populație combinând indivizi folosind un proces care se bazează pe o selecție (selection) după o anumită distribuție de probabilitate bazată pe funcția de fitness

Selecția proporțională

Dezavantaje:

1. Dacă funcția de optimizat poate lua valori negative, atunci valorile de fitness trebuie transformate astfel încât să fie strict pozitive (spre exemplu adăugând o constantă sau folosind funcția ***softmax***).





2. Dacă diferența de fitness între cel mai bun individ și restul populației este foarte mare, selecția poate deveni excesiv de concentrată pe descendenții acestuia. Astfel, procesul evolutiv poate stagna, iar lipsa diversității face dificilă explorarea de noi soluții.

Selecția pe baza rangurilor

Idee:

Probabilitatea de selecție nu depinde de valorile absolute ale funcției de fitness, ci de poziția fiecărei valori în clasamentul ordonat crescător.

Etape:

1. Ordonează crescător valorile funcției de fitness.
2. Fiecărei valori distincte din lista de valori i se asociază un rang (rangul 1 pentru cea mai mică valoare).
3. Se partiționează populația în clase de elemente având asociat același rang (de exemplu, k clase).
4. Se calculează probabilitățile de selecție: $P_i = i / (1+2+\ldots+k)$.
5. Se selectează clase de elemente utilizând tehnica ruletei sau SUS; din fiecare clasă selectată se extrage aleator câte un element.

Selecția de tip turneu

Idee:
Selecția fiecărui individ se face prin compararea lui cu un grup mic de indivizi aleși aleator din populație.

Etape:
1. Se selectează aleator k indivizi.
2. Dintre cei k indivizi, se alege cel cu cel mai bun fitness, acesta este câștigătorul turneului.

Observații:
1. Turneele pot fi cu sau fără revenire (cu sau fără posibilitatea de a selecta același individ de două ori).
2. Caz frecvent utilizat: k = 2 (turneu binar).

Selecția prin trunchiere

Idee:
Din populația formată din părinți și urmași se selectează direct cei mai buni k indivizi.

Observații:
1. Este singura metodă de selecție deterministă, nu implică aleatoriu.
2. Este folosită mai des în strategiile evolutive.
3. Dacă atât părinții, cât și urmașii conțin câte m elemente, atunci trebuie selectați m supraviețuitori dintre cele 2m elemente.

Avantaj: presiune puternică asupra îmbunătățirii calității populației.
Risc: scăderea rapidă a diversității.

Încrucișare

Scop:
Combinarea informației genetice a doi (sau mai mulți) părinți pentru a produce indivizi noi, potențial superiori.

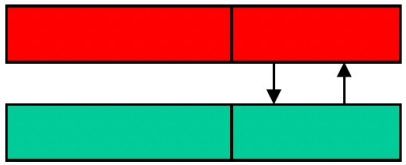
Observație:
În algoritmii genetici clasici se pornește de la doi părinți și se generează doi urmași.

Tipuri de încrucișare:
- cu unu sau mai multe puncte de tăietură
- uniformă
- convexă
- metode dependente de problema rezolvată

Încrucișare cu puncte de tăietură

Un punct de tăietură

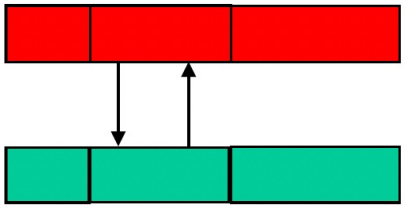
Părinți



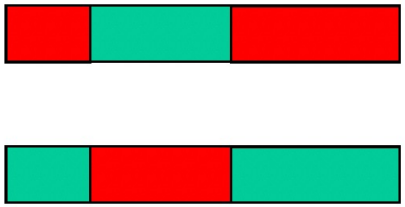
Urmași

Două puncte de tăietură

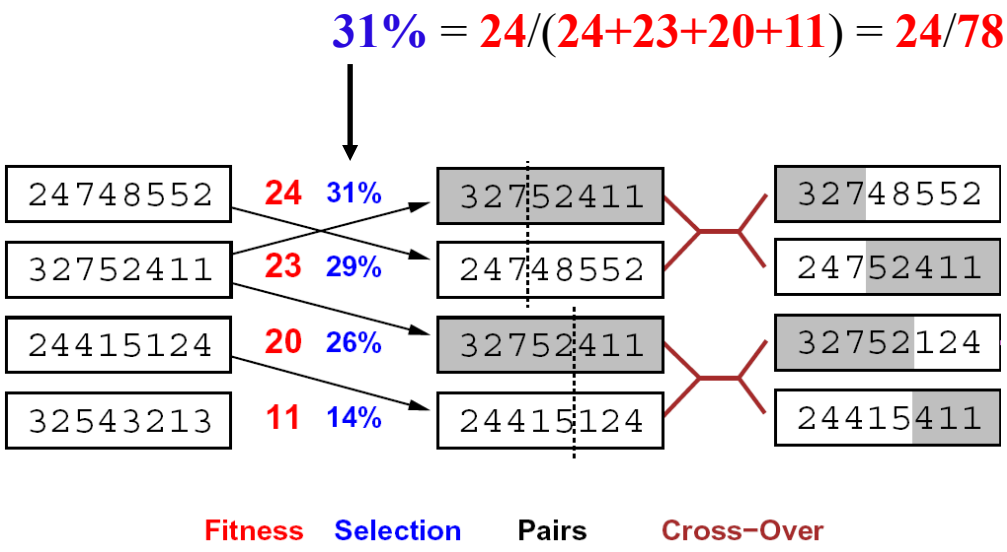
Părinți



Urmași



Încrucișare cu puncte de tăietură



- Obține o nouă populație combinând indivizi folosind un proces care se bazează pe o selecție (selection) după o anumită distribuție de probabilitate bazată pe funcția de evaluare (fitness) si împerechere (**pairs**) bazată pe operațiile de **încrucișare (cross-over)** și mutație (mutation)

Mutație

Scop:
Introducerea unor gene noi în populație, menținând diversitatea și evitând blocarea în optime locale.

Observație:
Tipul de mutație depinde de codificarea cromozomilor.

Exemplu - codificare binară:
Mutația constă în complementarea unor gene selectate aleator.

Variante:
1. Mutație la nivel de cromozom
2. Mutație la nivelul ansamblului de gene

Mutație la nivel de cromozom

Etape:

1. Se parcurge populația; pentru fiecare cromozom se decide dacă suferă mutație (probabilitatea de mutație P_m este de regulă o valoare foarte mică).
2. Dacă cromozomul este ales pentru mutație, se selectează aleator poziția genei care va fi modificată.

Observație:

Pentru a avea, în medie, o genă modificată pe cromozom, se poate alege probabilitatea de mutație $P_m = 1/m$.

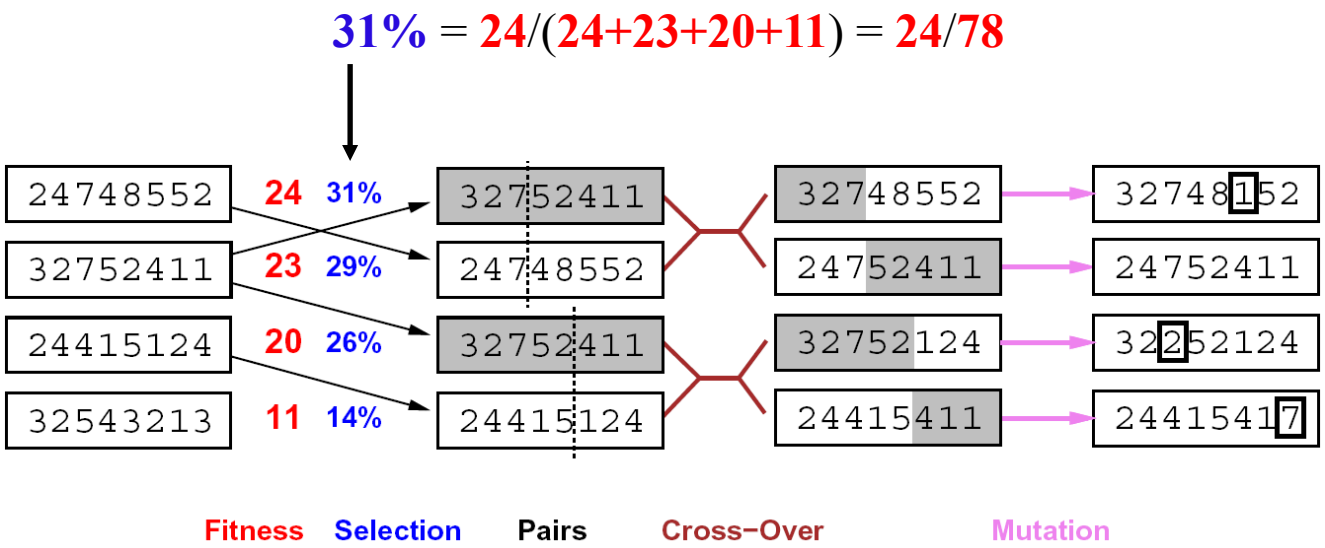
Mutație la nivelul ansamblului de gene

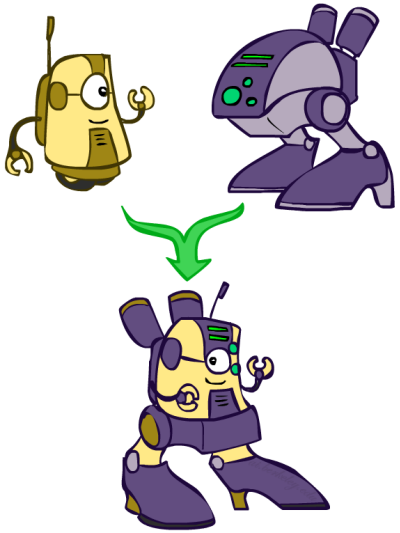
Ipoteza:
Toate elementele populației sunt privite ca un singur șir binar lung.

Aplicare:
Se parcurg toate genele și pentru fiecare se decide (cu probabilitatea P_m) dacă se complementează sau nu.

Observație:
În această variantă pot fi modificate mai multe gene ale unui cromozom.

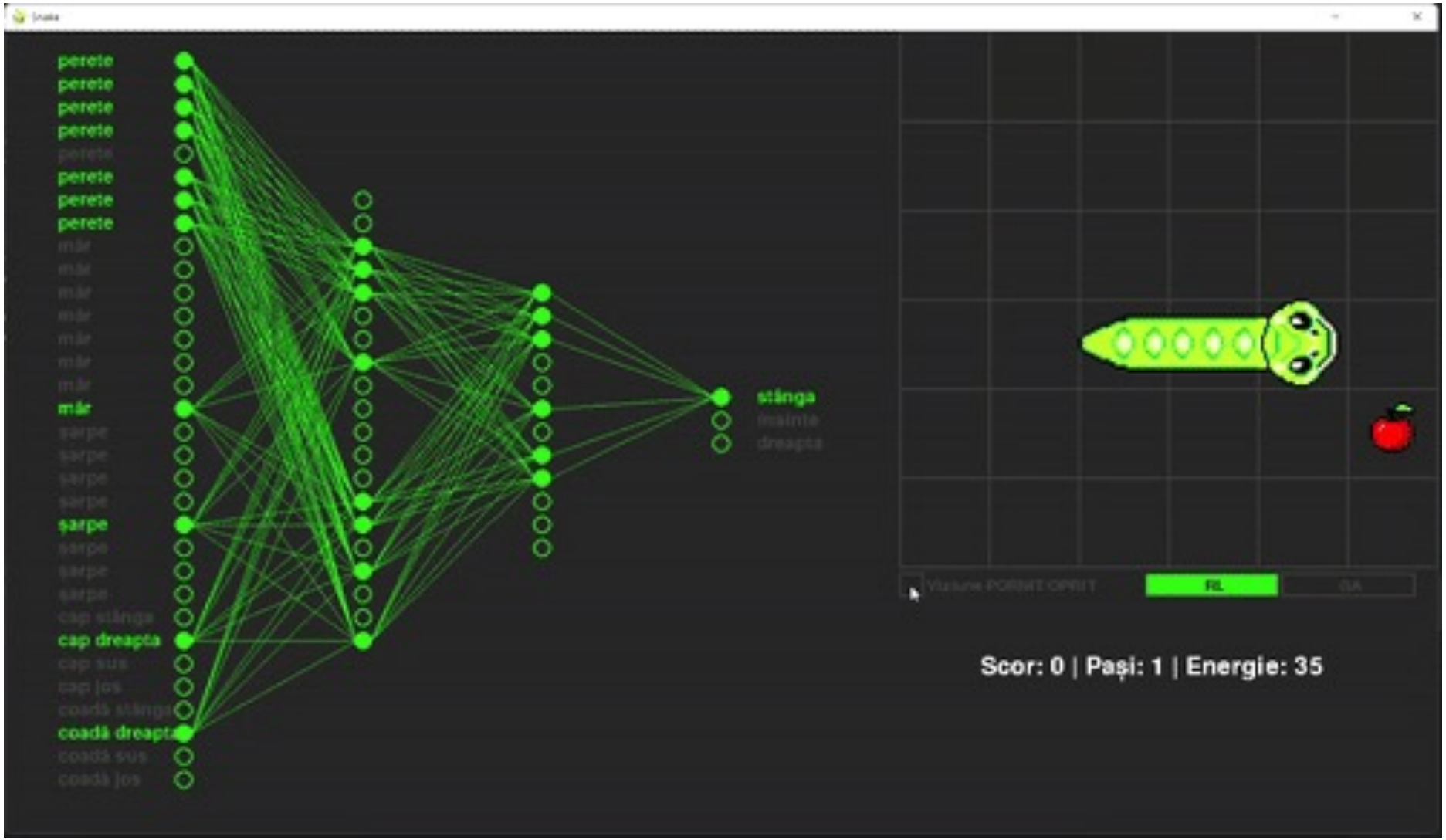
Mutația în algoritmi genetici





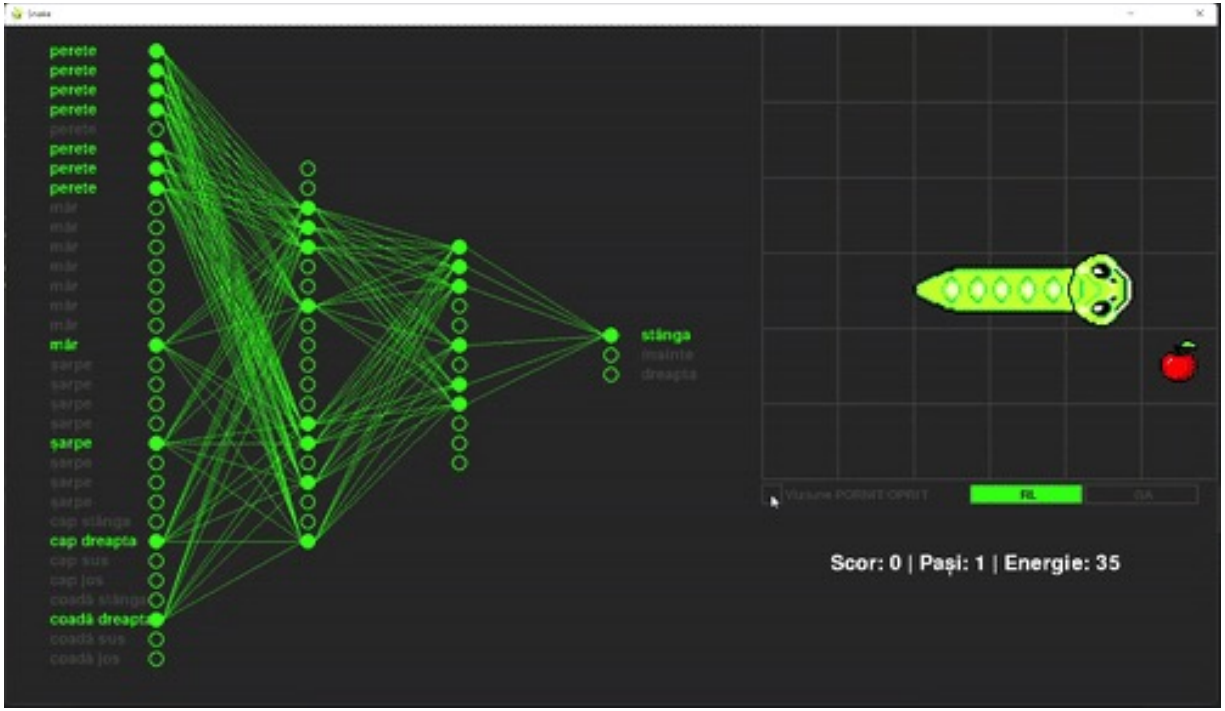
- Obține o nouă populație combinând indivizi folosind un proces care se bazează pe o selecție (selection) după o anumită distribuție de probabilitate bazată pe funcția de evaluare (fitness) si împerechere (**pairs**) bazată pe operațiile de **încrucișare (cross-over)** și mutație (mutation)

Algoritmi genetici – exemplu Snake

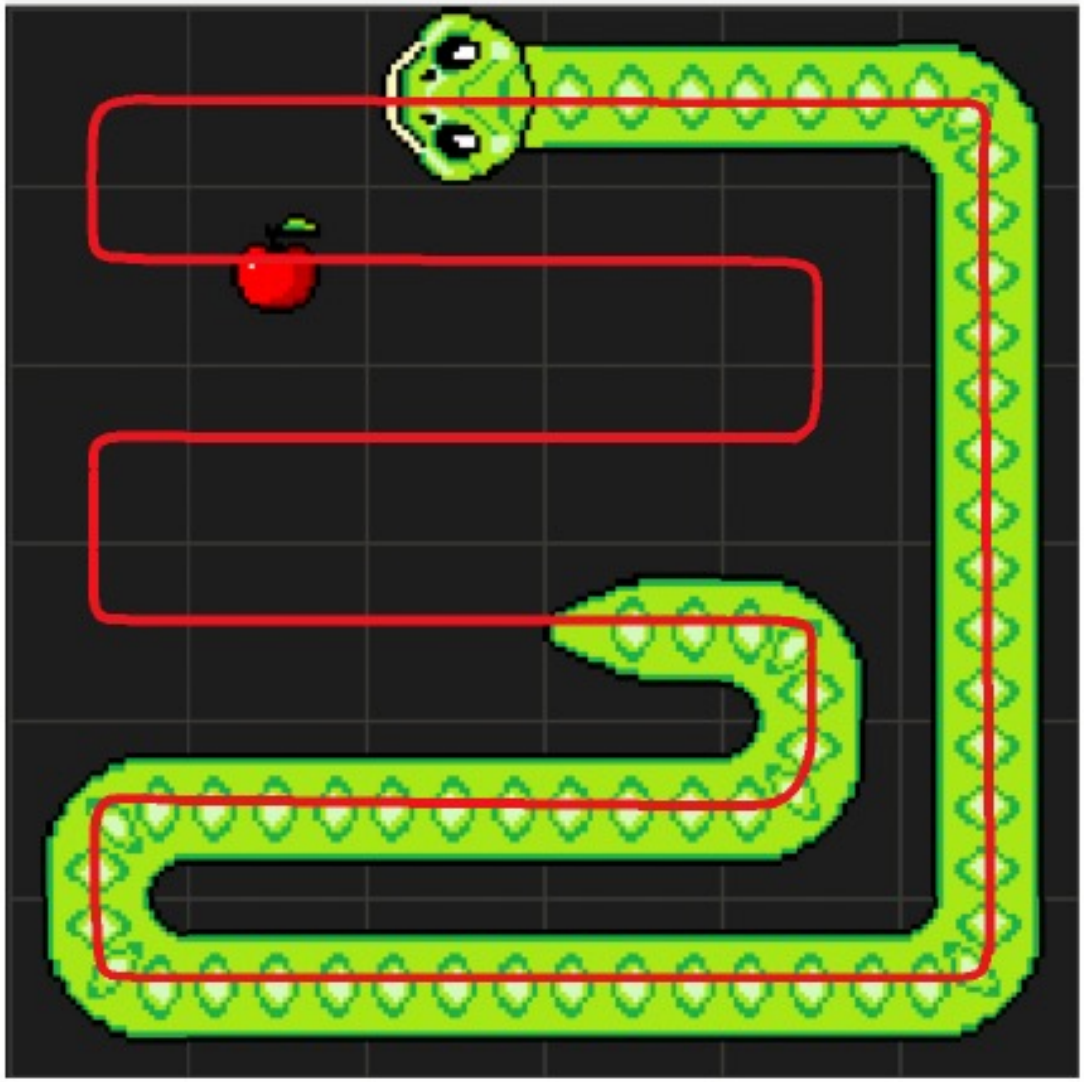


Snake - reguli de bază

- Șarpele se deplasează într-una dintre cele patru direcții: sus, jos, stânga, dreapta.
- Coliziunea cu pereții sau cu propriul corp duce la încheierea jocului.
- Fiecare măr consumat adaugă un segment la coada șarpelui și crește scorul cu 1.
- Scor = număr de mere mâncate (maxim 36 = grid 6×6 – 3 lungime inițială snake)



Soluție naivă



Eșecuri

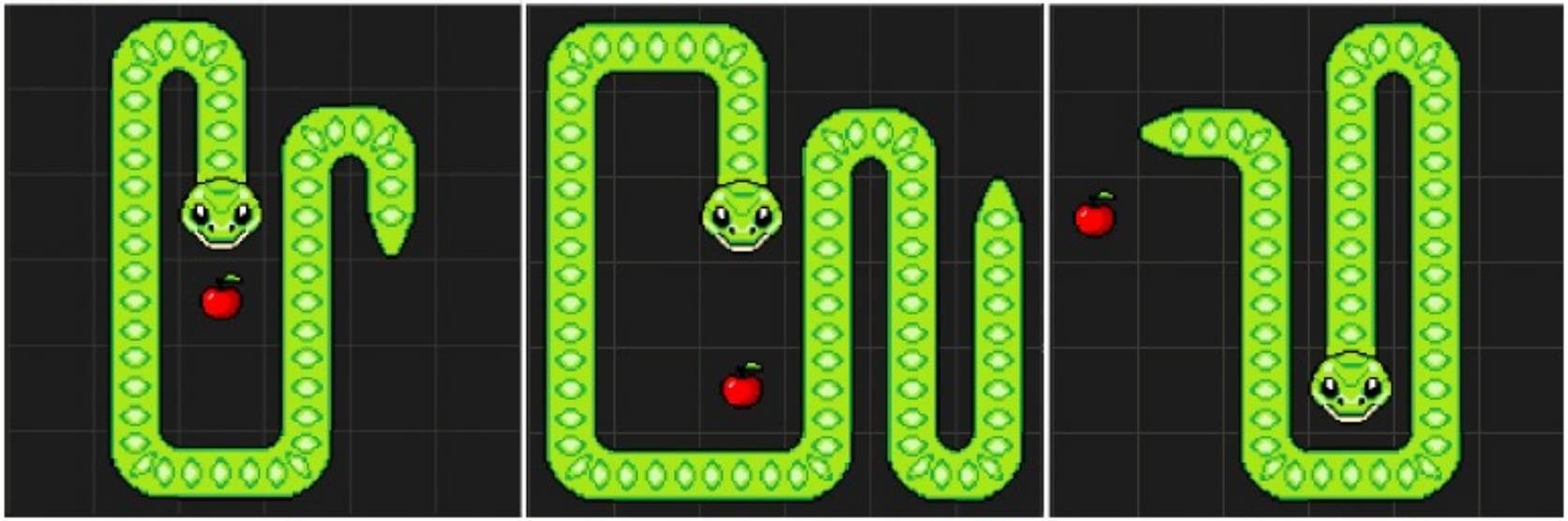
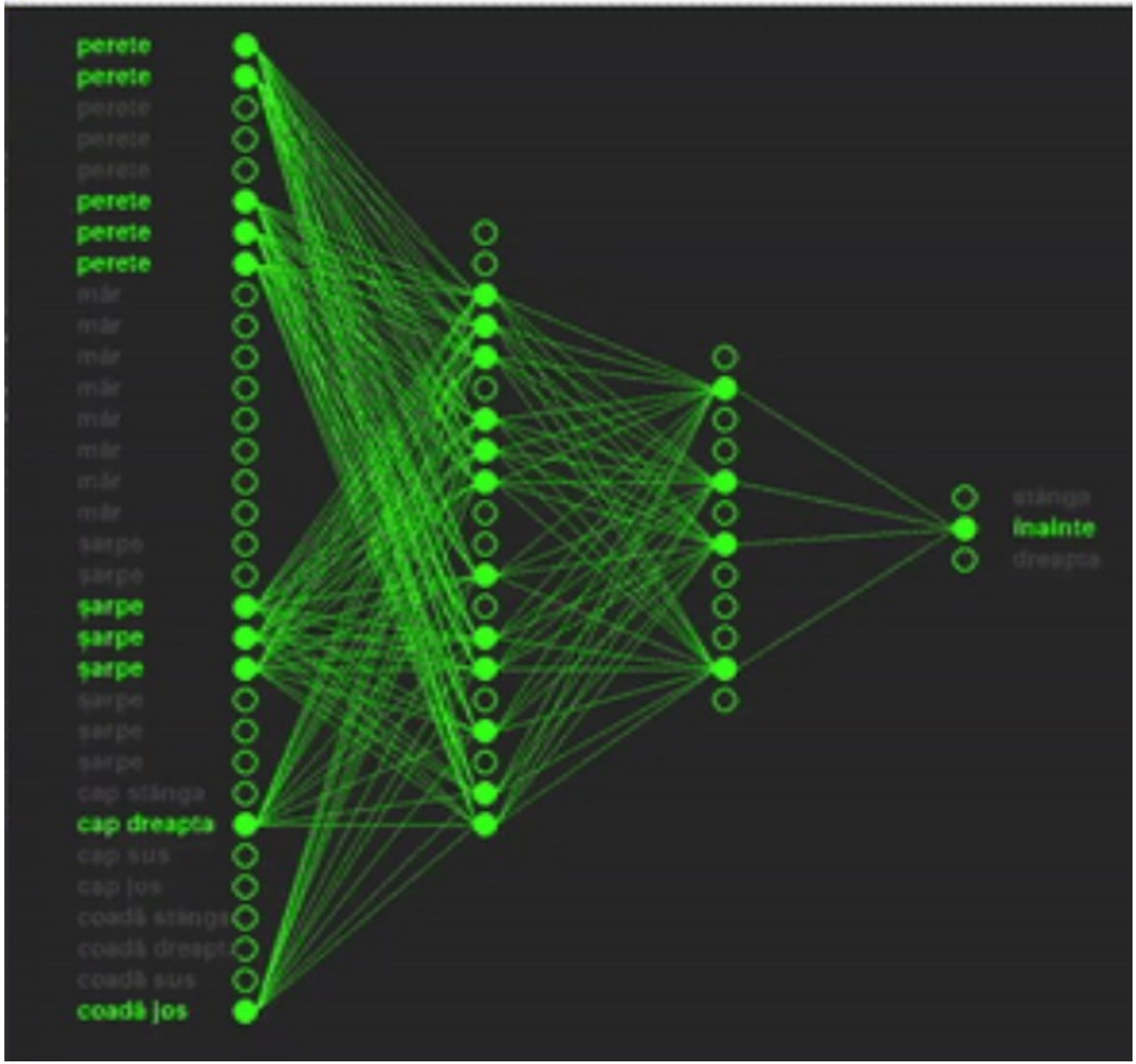


Figura 3.3: Situații în care șarpele ajunge blocat în propriul corp

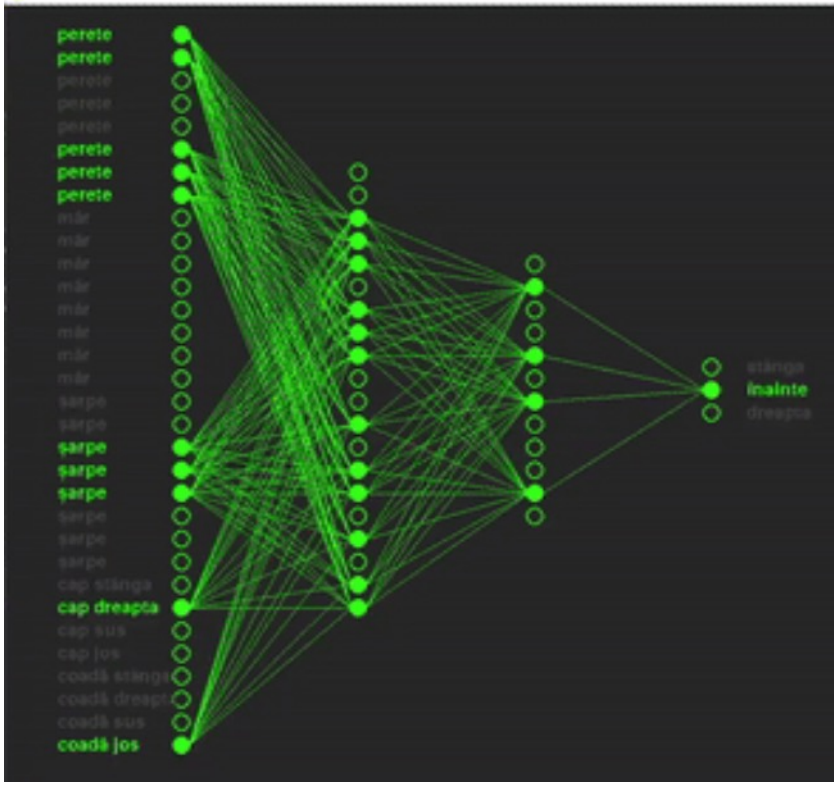
Snake – arhitectura rețelei



Snake – arthitectura rețelei

Rețeaua neuronală este un Multi-Layer Perceptron de tip feedforward. Aceasta este compusă din:

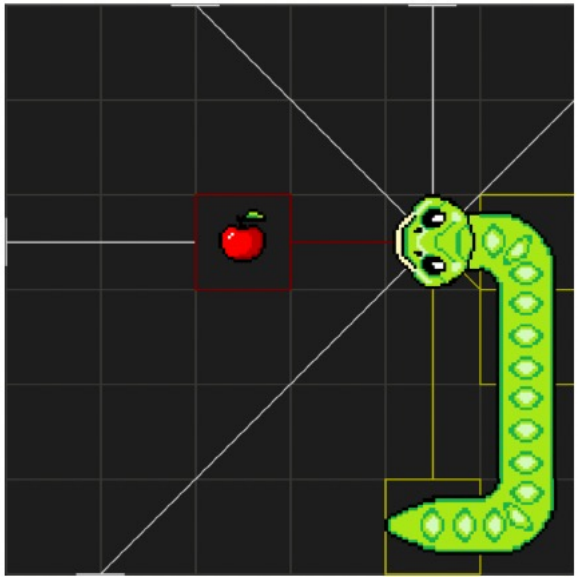
- Strat de intrare: 32 de neuroni, corespunzând vectorului de stare al șarpelui;
- Strat ascuns 1: 20 de neuroni
- Strat ascuns 2: 12 de neuroni
- Strat de ieșire: 3 neuroni, corespunzători celor trei acțiuni posibile: stânga, înainte și dreapta.



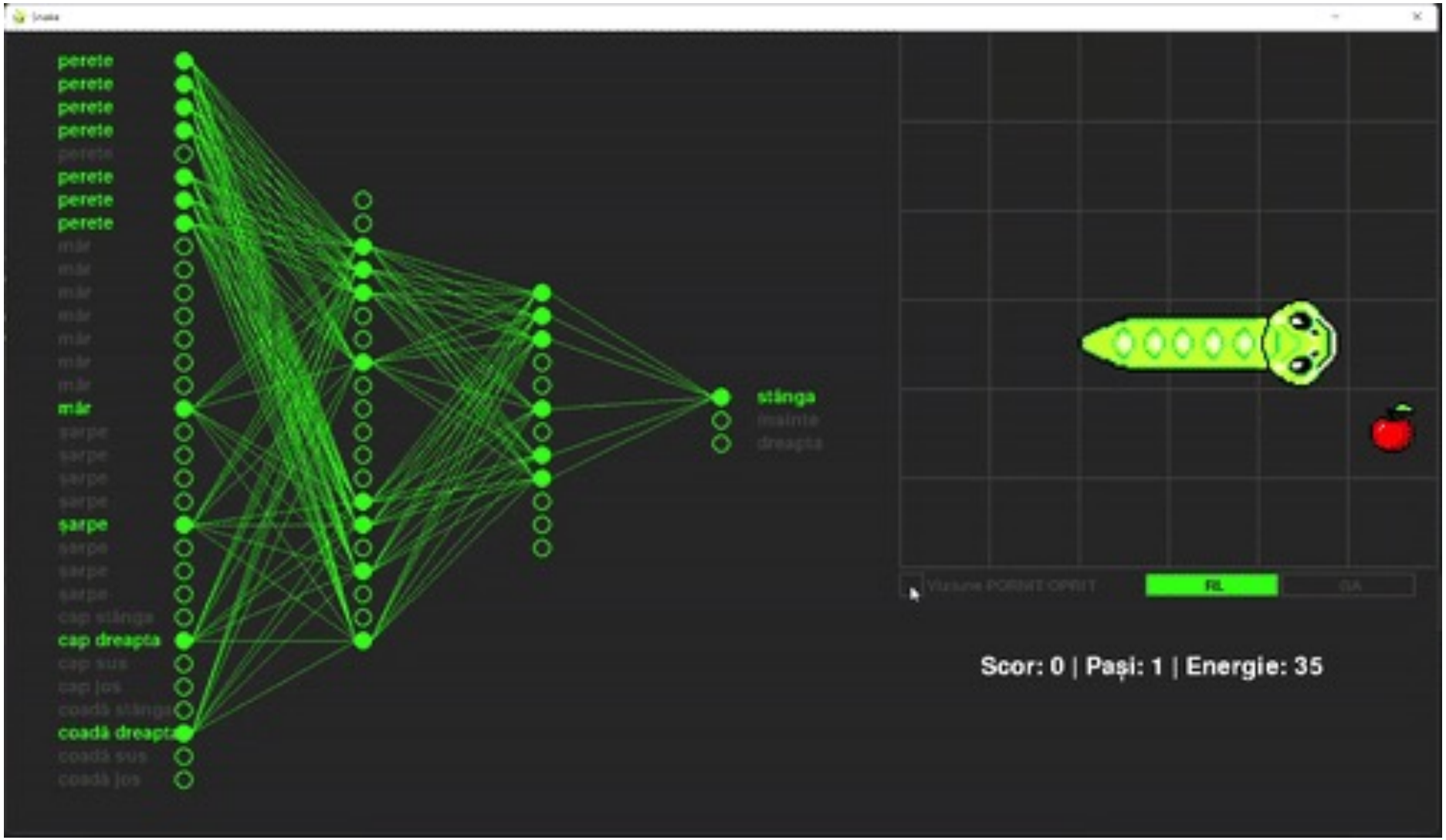
Datele de intrare – starea curentă

Datele de intrare ale rețelei neuronale sunt extrase din mediul jocului și oferă o reprezentare compactă a stării curente a șarpelui. Vectorul de intrare are dimensiunea 32 și este alcătuit din următoarele componente:

- 8 valori care indică distanța până la perete în 8 direcții radiale;
- 8 valori care indică distanța până la corpul șarpelui în cele 8 direcții;
- 8 valori care indică prezența unui măr în cele 8 direcții;
- 4 valori care codifică direcția curentă a capului șarpelui ([sus, jos, stânga, dreapta]);
- 4 valori care codifică direcția cozii șarpelui, în același format ([sus, jos, stânga, dreapta]).



Datele de intrare – starea curentă



Datele de ieșire – decizia luată

Ieșirea rețelei neuronale constă într-un vector de dimensiune 3, în care fiecare componentă reprezintă probabilitatea asociată uneia dintre cele trei acțiuni posibile:

- viraj la stânga față de direcția curentă;
- menținerea direcției curente;
- viraj la dreapta față de direcția curentă.

Selectarea acțiunii se realizează pe baza valorii maxime din acest vector, corespunzător celei mai probabile acțiuni conform predicției rețelei. Astfel, dacă prima valoare este maximă, agentul va executa un viraj la stânga; dacă valoarea maximă se află pe poziția a doua, se menține direcția curentă; iar dacă a treia valoare este dominantă, se efectuează un viraj la dreapta.

perete
perete
perete
perete
perete
mar
mar
mar

Selecția

Selecția se bazează pe scorul de fitness: un număr fix de indivizi cu cele mai bune performanțe sunt păstrați neschimbați în generația următoare. Această strategie elitistă conservă caracteristicile eficiente și reduce riscul de pierdere a indivizilor performanți în urma mutațiilor.

Funcția de fitness

Pentru a determina calitatea unui individ, acesta este evaluat la finalul unei sesiuni complete de joc. Se calculează un scor de fitness care ia în considerare numărul total de pași supraviețuiți și numărul de mere consumate. Formula folosită este:

$$F = S + \left(2^A + A^{2.1} \cdot 500\right) - \left((0.25 \cdot S)^{1.3} \cdot A^{1.2}\right)$$

- F este scorul de fitness final,
- S reprezintă numărul total de pași,
- A reprezintă numărul de mere colectate.

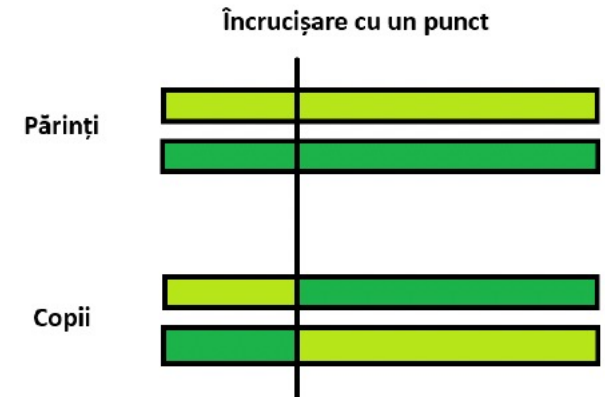
Scorul de fitness penalizează stagnarea sau mișcările repetitive care nu contribuie la colectarea de mere și recompensează comportamentele care implică explorare eficientă și maximizarea lungimii.

Încrucișarea

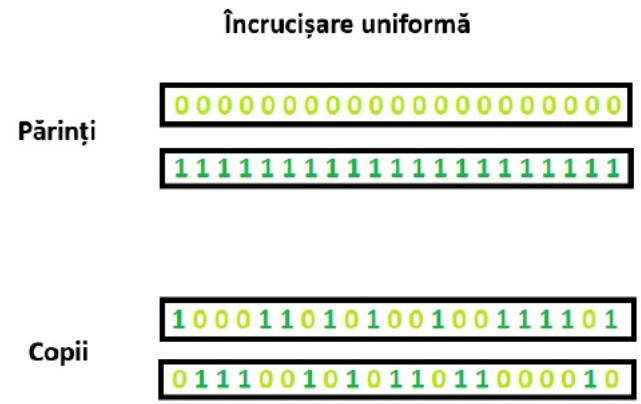
Încrucișarea are ca scop generarea de urmași prin combinarea informației genetice (parametrii rețelelor) de la doi părinți selectați. Încrucișarea se aplică direct asupra matricelor de ponderi și vectorilor de bias, care sunt reprezentați ca tablouri de numere reale.

Se utilizează două metode:

• Încrucișare binară cu un punct: se alege aleator o poziție în vectorul de parametri (sau într-o matrice), iar componentele sunt schimbate între părinți de la acel punct înainte.



• Încrucișare uniformă: fiecare parametru este selectat aleator de la unul dintre cei doi părinți.



Mutația

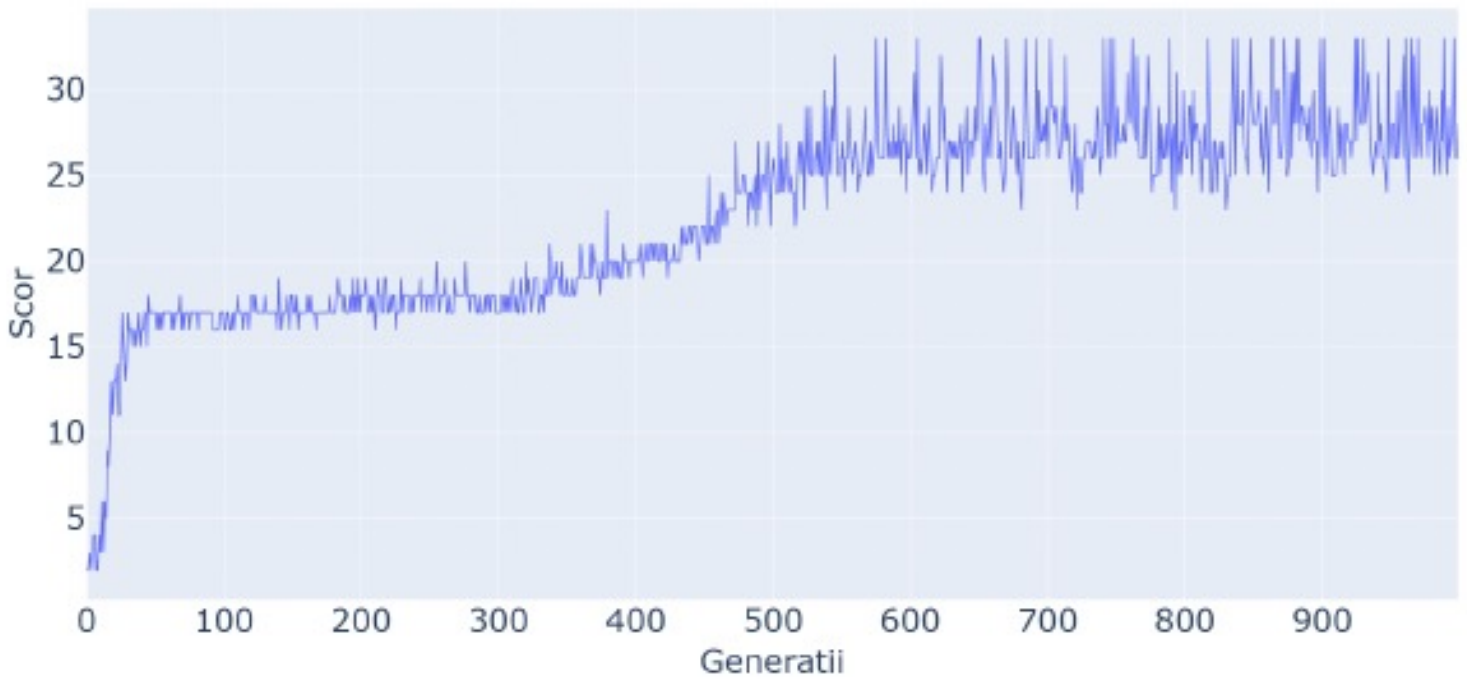
Mutația introduce diversitate în populație prin modificarea aleatoare a valorilor parametrilor descendenților. O posibilă alegere este mutație gaussiană, prin care fiecare element numeric al ponderilor și biasurilor poate fi ajustat prin adăugarea unui zgomot generat dintr-o distribuție normală:

$$\theta' = \theta + \mathcal{N}(0, \sigma)$$

Rolul mutației:

- introduce diversitate;
- previne blocarea într-un minim local;
- permite explorarea unor regiuni complet noi ale spațiului de strategii.

Evoluția scorului celui mai bun individ

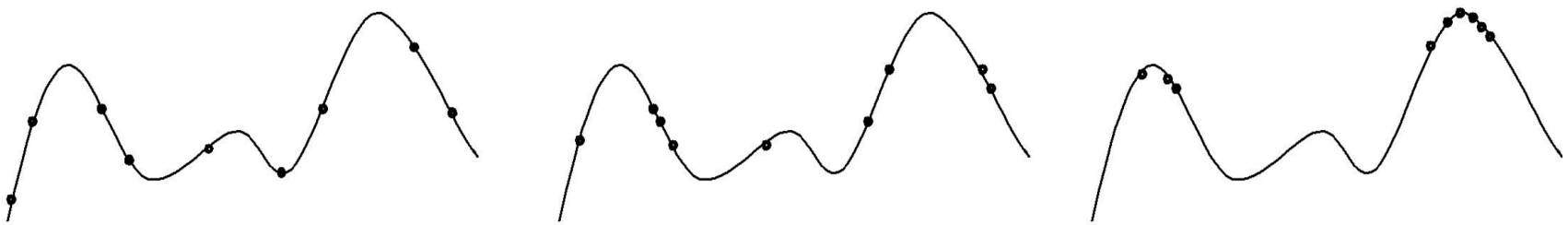


Scor = numărul total de pătrățele de pe hartă ocupate de șarpe minus lungimea inițială a șarpelui (3 segmente), ceea ce, în cadrul testării pe o hartă de dimensiune 6 × 6, corespunde unui scor de 33.

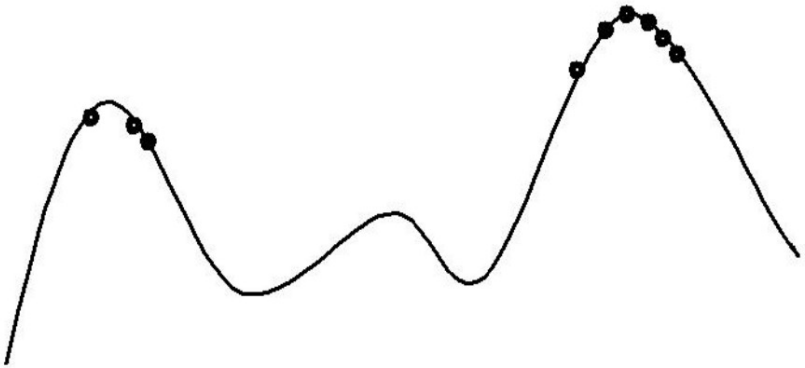
AI Learns:

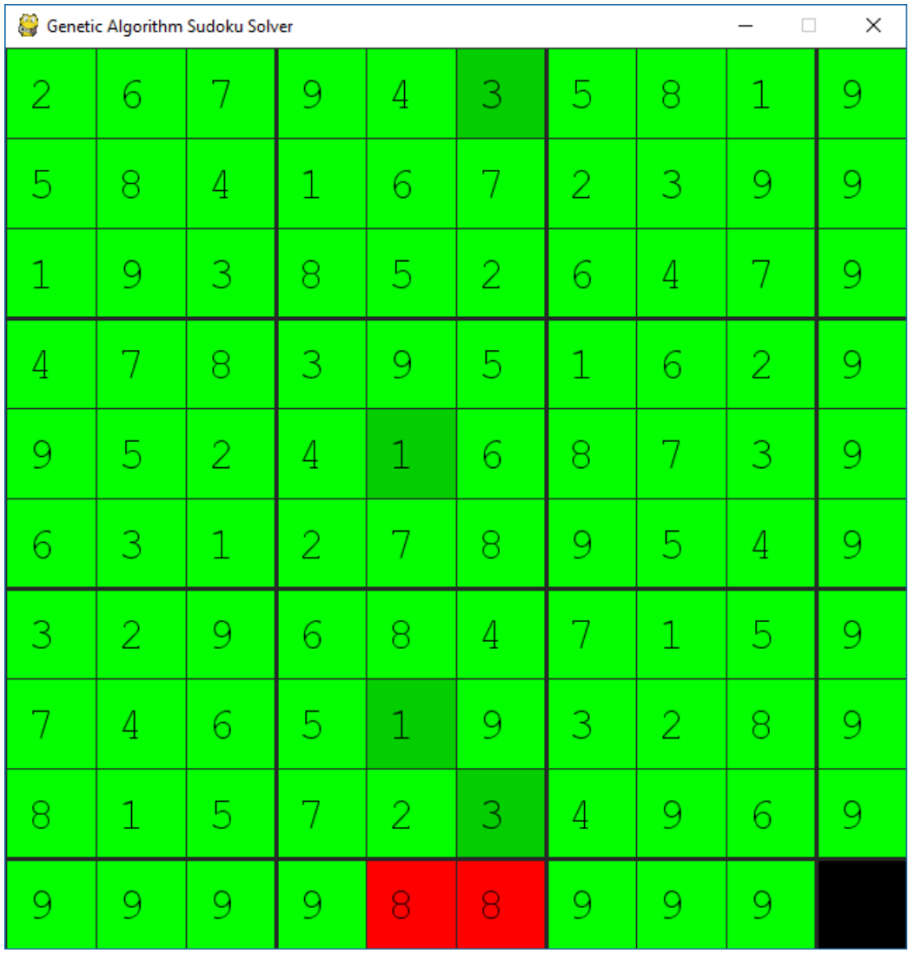
https://www.youtube.com/watch?v=vhiO4WsHA6

Comportamentul tipic al unui algoritm genetic

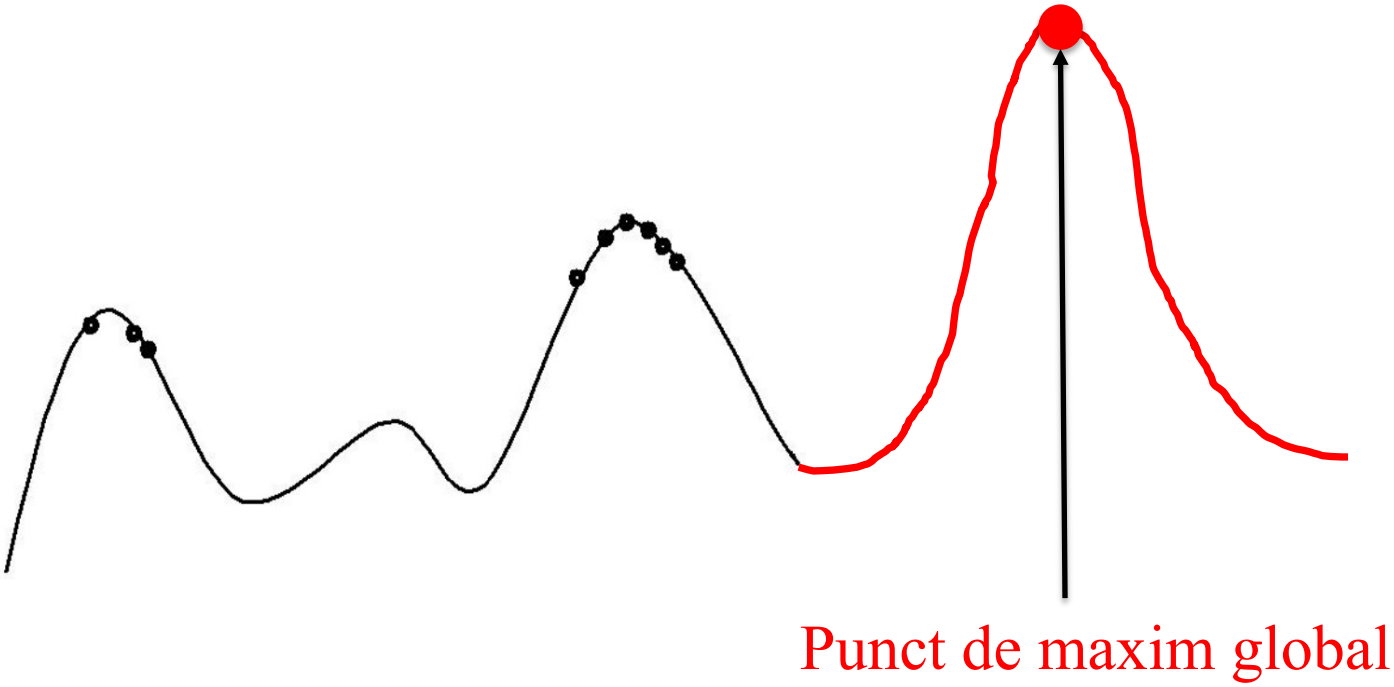


Maxime/minime locale

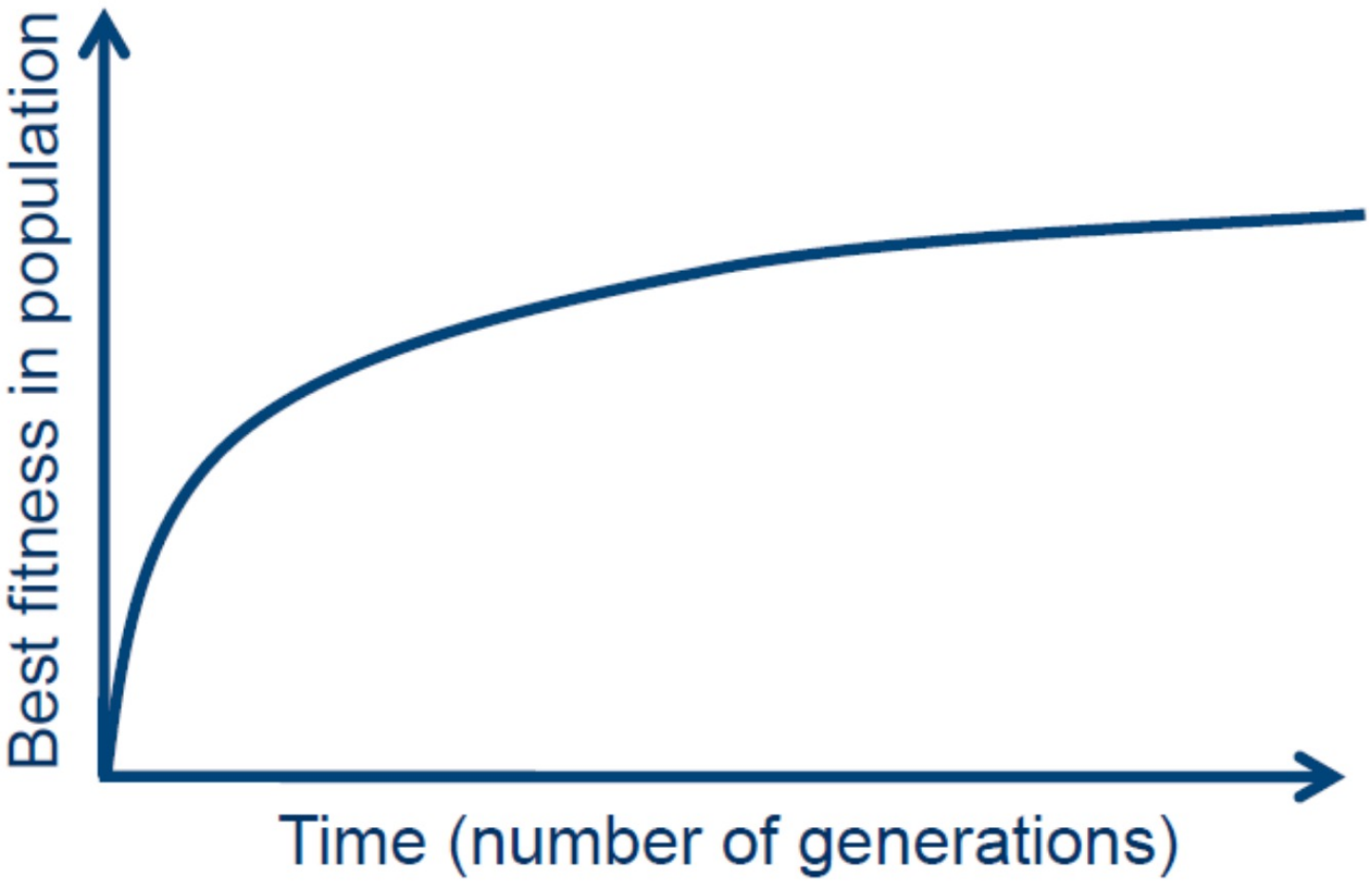
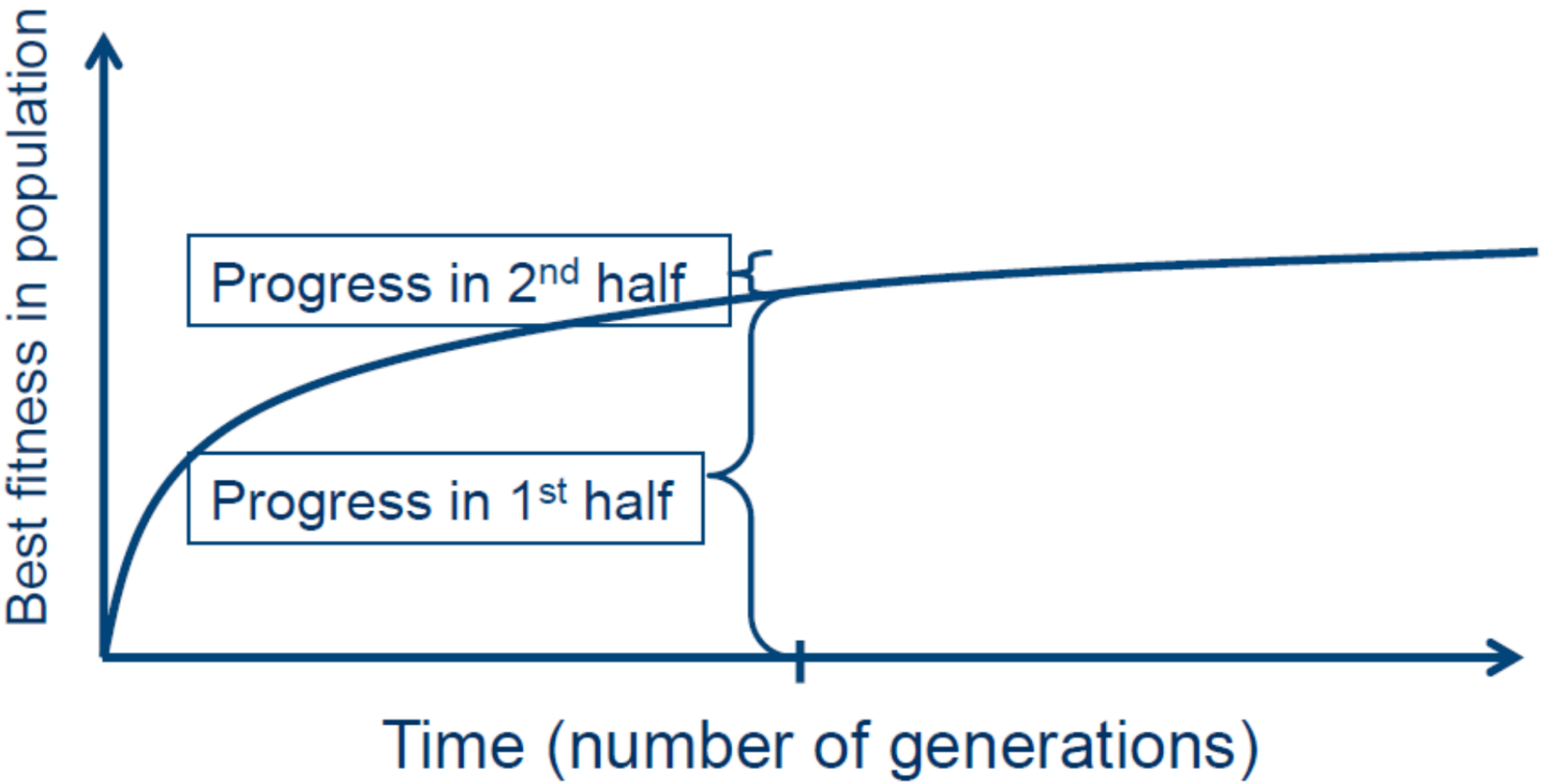




Maxime/minime globale



Evoluția tipică a unui algoritm genetic



Timp de rulare vs. fitness